RCA Report

Root Cause Analysis (RCA) Report
Title: Windows Server 2022 VM Memory Performance Degradation (Controlled Fault Injection)
Date: 2026-08-12
Environment: Non-production VM only
Prepared by: Infrastructure/SRE Team
Status: Draft for validation run evidence

1) Executive Summary
A controlled memory-pressure scenario was prepared to simulate VM memory performance degradation on Windows Server 2022. The objective was to validate observability, rollback readiness, and recovery criteria before introducing the fault. The design enforces a safety-first sequence: baseline assessment, restore validation, then fault injection. The incident class under analysis is reduced memory availability with increased paging and latency for non-critical workloads.

2) Scope and Safety Boundaries
In scope:
- Memory pressure in a single non-production Windows Server 2022 VM
- Non-critical workload response time degradation
- Paging behavior and committed memory growth

Out of scope:
- Domain controllers
- Clustered workloads
- Production-critical applications
- Kernel pool exhaustion, crash dump generation, bug checks

Safety controls implemented:
- Hard usage ceilings and free-memory floors
- Time-bounded fault duration with auto-restore behavior
- Dedicated restore workflow and success criteria prior to faulting
- No OS configuration mutation (no registry, service, or page-file policy changes)

3) Pre-Change Assessment (Deliverable A)
Assessment script: scripts/memory-assess.ps1
Captured baseline domains:
- Current memory utilization (total/used/free physical memory)
- Available memory (Available MBytes and FreePhysicalMemory)
- Page file status (allocated/current/peak and settings)
- Running services inventory
- Performance baseline (30-second counter averages)
- VM configuration details (OS build, CPU, NUMA, Hyper-V guest indicator)

Evidence artifacts to attach:
- Baseline report JSON: %TEMP%\MemoryAssessment\baseline_<timestamp>.json

4) Incident Trigger and Detection
Trigger type:
- Intentional and controlled memory pressure simulation

Detection signals expected:
- Rising % Committed Bytes In Use
- Falling Available MBytes
- Increased Page Reads/sec and Page Faults/sec
- Increased response times on non-critical workloads

Primary data sources:
- Windows performance counters
- Baseline and restore JSON reports
- Local script console telemetry

5) Timeline (Fill with run timestamps)
T0 Baseline captured via memory-assess.ps1
T1 Restore dry-run executed via memory-restore.ps1 -DryRun
T2 Restore live validation executed via memory-restore.ps1
T3 Fault injection started via memory-fault.ps1
T4 Memory pressure plateau reached and monitored
T5 Auto-restore timeout reached or manual restore initiated
T6 Recovery validation completed and report written

6) Technical Root Cause
Primary cause:
- Artificial user-mode memory commitment from multiple worker processes consumed a significant portion of available physical memory, forcing higher paging activity and reducing memory headroom for concurrent workloads.

Mechanism:
- Child PowerShell processes allocate large byte arrays and periodically touch pages, maintaining residency pressure.
- As available memory shrinks, the VM relies more on page-file activity, increasing latency for memory-competing processes.

Why this produced degradation:
- Memory contention increased page transitions and disk-backed page reads.
- Application response paths requiring memory allocation or page-in operations experienced slower completion times.

7) Contributing Factors
- VM memory size and overcommit posture determine how quickly thresholds are reached.
- Existing baseline memory utilization affects safe injection headroom.
- Page file size and storage performance influence paging latency magnitude.
- Number of worker processes and pressure target impact fault intensity.

8) Rollback Strategy (Deliverable B)
Restore script: scripts/memory-restore.ps1

State restored:
- Terminates fault worker processes
- Removes sentinel state file used by workers
- Re-validates post-restore memory counters

Recovery prerequisites:
- Elevated shell (Administrator)
- Access to local process table
- Optional: baseline JSON from assessment for comparative validation

Rollback success criteria:
- No fault worker processes remaining
- Available memory >= configured threshold (default 512 MB)
- % Committed Bytes In Use below safety bound (default < 85%)
- Page Reads/sec returned near baseline tolerance (<= 2x baseline when baseline supplied)

Estimated recovery validation time:
- Process termination: < 5 seconds typical
- Reclaim and stabilization: 10-60 seconds typical
- Validation sampling window: default 60 seconds
- End-to-end expected: ~2-5 minutes

9) Recovery Validation Results (Populate after execution)
Restore report artifact:
- %TEMP%\MemoryAssessment\restore_<timestamp>.json

Record observed values:
- Available MBytes (post-restore avg): <insert>
- % Committed Bytes In Use (post-restore avg): <insert>
- Page Reads/sec (post-restore avg): <insert>
- Page Faults/sec (post-restore avg): <insert>
- AllCriteriaPassed: <true/false>

Outcome classification:
- Recovered: All criteria true
- Partial recovery: One or more criteria false; rerun restore and investigate workload pressure
- Failed recovery: Persistent low memory or high paging after repeated restore attempts

10) Business/Service Impact Assessment
Expected impact window:
- Short-lived and isolated to non-critical workloads in test VM

Potential effects during fault window:
- Slower response times
- Increased queueing and intermittent timeouts in non-critical services
- Elevated paging activity and CPU overhead from memory management

No expected impacts:
- No data corruption by design

- No OS configuration drift by design
- No deliberate crash/bugcheck path

11) Corrective and Preventive Actions (CAPA)
Immediate actions:
- Enforce execution order: assess -> restore validate -> fault inject
- Require baseline and restore report artifacts for each run
- Keep restore terminal ready before fault start

Engineering improvements:
- Add explicit runbook gate checks in CI or orchestration wrapper
- Add automatic export of perf counters to a single incident bundle
- Add workload-specific SLO probe during fault window for measurable latency impact
- Add guardrails to block execution on tagged critical hosts

Operational controls:
- Change record approval for all non-prod chaos tests
- Mandatory maintenance window for memory pressure exercises
- Post-run review of thresholds and tuning before next test cycle

12) Lessons Learned
- Recovery-first design significantly lowers operational risk during fault experiments.
- Baseline quality directly affects confidence in recovery assertions.
- Thresholds must be environment-specific to avoid over-aggressive pressure in small-memory VMs.

13) Final RCA Statement
This event class is expected and controllable when generated by the approved memory fault script under configured safety thresholds. The root cause of observed degradation is intentional user-mode memory pressure leading to reduced available memory and increased paging. Recovery is achieved by terminating fault workers, removing sentinel state, and validating counter normalization against predefined criteria.

14) Approval and Sign-off
Incident Owner: <name>
SRE Reviewer: <name>
Infrastructure Approver: <name>
Date Closed: <yyyy-mm-dd>

Appendix A: Relevant scripts in repository
- scripts/memory-assess.ps1
- scripts/memory-restore.ps1
- scripts/memory-fault.ps1

Appendix B: Recommended execution order
1. .\memory-assess.ps1
2. .\memory-restore.ps1 -DryRun
3. .\memory-restore.ps1
4. .\memory-fault.ps1 -TargetPressureMB 1024 -MaxDurationSeconds 300

5. .\memory-restore.ps1 -BaselineReport "%TEMP%\MemoryAssessment\baseline_<timestamp>.json"